PROJECT OVERVIEW

slideotter

A local browser studio for planning, editing, reviewing, validating, and archiving structured decks.

This tour shows new users where the work lives and how an idea becomes an archived PDF.

Structured source

Slides, context, materials, and state stay inspectable in the project tree.

Live workbench

Preview, edit, attach images, compare variants, and plan decks in one browser flow.

Guarded output

Generated changes wait for review, then validation protects the archive.

TOUR MAP

What you will learn

The deck follows the path a new author takes: choose a presentation, shape the content, review changes, and ship a checked archive.

Start

Pick or create a presentation, then give it useful constraints.

Author

Edit structured slides, attach materials, and keep context close.

Review and ship

Compare candidates, run checks, and archive the intended version.

The same concepts apply whether you are editing the included demo or creating a new deck.

PROBLEM

Why decks get messy

Deck work gets hard when source, preview, and final PDF stop agreeing.

What usually drifts

Source
The file no longer explains the visible slide.

Context
Audience and tone live outside the deck.

Options
Ideas replace originals before review.

Output
PDF issues appear after export.

What slideotter keeps together

One local loop
Edit, preview, compare, and validate in the studio.

Visible source
Keep JSON close to the rendered slide.

Intentional archive
Validate before archiving.

WORKFLOW

A simple authoring loop

Most work follows four steps, and each step leaves readable local state behind.

Daily flow

1. **Plan**
   Capture objective, outline, audience, constraints.
2. **Edit**
   Change text or JSON beside the preview.
3. **Review**
   Compare alternatives before applying.
4. **Ship**
   Build, validate, archive.

Why it stays calm

Local
The studio runs against this repository.

Explicit
Buttons request bounded server actions.

Inspectable
State remains readable on disk.

WORKSPACE

Choose a presentation first

slideotter can manage more than one deck. The selector shows the first slide, name, and facts before you open it.

What a deck owns

Slides
Specs under presentations/<id>/slides.

Context
Brief, outline, theme, checks.

Materials
Images and metadata beside the deck.

State
Variants, history, local choices.

What you can do there

Create
Describe constraints before the scaffold appears.

Duplicate
Copy a deck when you want a branch.

Delete
Remove a deck only after confirmation.

CONTEXT

Give the deck a brief

Before generating or reorganizing, write what the deck should do.
Studio workflows reuse that context.

Useful brief fields

Audience

Who needs to understand or decide?

Objective
What should become clearer?

Tone
Direct, technical, executive, tutorial.

Constraints
Non-negotiables for words and visuals.

Where the brief helps

Planning
Deck plans use the saved brief and outline.

Variants
Generated options can respect the same intent.

Checks
Validation settings travel with the deck.

SOURCE MODEL

Slides are readable JSON

Each structured slide is a compact JSON file. The renderer turns those fields into the slide you inspect in the studio.

Current slide families

Cover
Identity, summary, logo, cards.

TOC
Tour framing and section cards.

Content
Evidence and guardrail panels.

Summary
Takeaways and resource cards.

Why JSON helps

Parseable
Invalid JSON cannot be saved.

Diffable
Changes stay reviewable in source.

Extensible
New families can be added later.

SLIDE STUDIO

Edit with the slide in view

The main preview is the working surface. Text and JSON edits update it immediately.

Edit path

1. **Select**
   Choose the target in the slide strip.
2. **Inline**
   Double-click editable preview text.
3. **JSON**
   Open the spec drawer.
4. **Save**
   Persist valid changes.

What prevents mistakes

Parse errors block save
Invalid JSON cannot persist.

Selected slide only
Actions stay scoped.

Preview stays central
Editing chrome is secondary.

MEDIA

Attach images as materials

Upload images into the active presentation, describe them, then attach one when it belongs on a slide.

Material flow

Upload
Files land in presentations/<id>/materials.

Describe
Metadata records alt text and caption.

Attach
JSON stores an optional media object.

Render
The DOM places image and caption together.

Practical rules

- **Alt text**
  Make the image understandable.
- **Small files**
  Server limits keep the repo manageable.
- **Explicit attach**
  Upload alone never edits a slide.

RUNTIME

One renderer keeps previews honest

The studio avoids separate preview and export logic. The same DOM slide powers authoring, comparison, validation, and PDF output.

Surfaces using it

Main preview
The large slide in Slide Studio.

Thumbnails
Selector and presentation cards.

Compare panes
Current and candidate views.

PDF export
Playwright prints the DOM output.

Why it matters

One path
Fewer places for layout drift.

Browser measured
Checks inspect rendered geometry.

Deck themed
Slides keep their own visual theme.

VARIANTS

Generate options, not surprises

Generation creates review candidates. It does not silently replace the current slide.

What you control

How many

Choose candidate count; five is default.

What kind

Try wording, structure, layout, or theme.

How generated

Use local, LLM, or auto mode.

How reviewed

Preview and diff every option.

Apply discipline

Session-only

Candidates stay outside slide files until applied.

Media preserved

Image attachments carry forward.

Author decides

Promotion always needs a command.

REVIEW

Compare before applying

When candidates exist, the compare workspace becomes the decision surface: current slide, candidate slide, changed fields, and apply buttons.

What to inspect

Current

The slide you would replace.

Candidate

The proposed version.

Changes

Touched fields and line diffs.

Apply

Promote with optional validation.

Decision cues

Hidden until useful

No empty comparison area.

Direct selection

Rows choose the candidate to inspect.

No surprise writes

Nothing changes without Apply.

DECK PLANNING

Plan the deck above slide level

Deck Planning is for outline, order, missing sections, and shared context, not single-slide edits.

Plan candidate contents

1. **Outline**
   Proposed order and section roles.
2. **Actions**
   Insert, retitle, replace, archive, move.
3. **Context**
   Optional brief, theme, or constraints patch.
4. **Preview**
   Before and proposed deck strips.

Promotion rules

Shared patch optional
Apply structure without context changes.

Paths are named
Review touched files.

Apply once
Plan promotion is explicit.

ASSISTANT

Use chat for bounded tasks

The chat panel launches workflows. It can use selected slide text and call known operations.

Inputs it sees

Message
The request you type.

Selection
Optional text from the preview.

Slide
Current slide id and context.

Mode
Local, LLM, or automatic generation.

How it stays inspectable

Known actions
Chat triggers studio workflows.

Local history
Sessions remain repo-local.

Debug panel
Provider state is hidden until needed.

SAFETY

Know what can change

The server keeps writes narrow. Browser actions request specific mutations instead of giving the UI broad filesystem access.

Allowed targets

Slide specs
Structured JSON in the active presentation.

Presentation state
Context, variants, materials, and preferences.

Materials
Image files for the active deck.

Generated output
Preview pages, PDFs, and archive copies.

Change discipline

- **Allowlisted paths**
  Unexpected targets are rejected.
- **Named actions**
  Add, remove, attach, compare, apply.
- **Separate archive**
  Sharing output is deliberate.

VALIDATION

Run checks before sharing

Validation is subtle in the UI, but it is the final safety net before an archived PDF becomes the version people see.

Gate coverage

Types
Studio, server, and scripts compile.

Layout
Rendered slide bounds, text, and contrast.

Media
Images, captions, and preservation.

Browser
Studio workflows and page chrome.

What failures mean

Baseline drift
Refresh only intentional visual changes.

Severity settings
Rules can be warning or error.

Fixtures
Known regressions stay covered.

OUTPUT

Archive the version you mean to share

Building and archiving are separate. Build creates current output; archive update copies the presentation PDF only when you intend to publish it.

Artifact path

1. **Build**
   slides/output/<presentation-id>.pdf.
2. **Render pages**
   studio/output/<id>/rendered-pages.
3. **Baseline**
   studio/baseline/<id>.
4. **Archive**
   archive/<presentation-id>.pdf.

Routine

Regenerate
Build after slide or theme changes.

Validate
Run the quality gate.

Archive
Update the checked-in snapshot.

FOR CONTRIBUTORS

How to read the project

If you are new to the codebase, start with the folder that matches the change.

Where to look

presentations/
Deck source, context, materials, state.

studio/
Browser app, server, renderer, baselines.

scripts/
Build, validation, archive commands.

docs/ and status
ADRs, roadmap, status notes.

Working habits

Small changes
Keep edits close to the boundary.

Docs follow code
Update guidance when workflows move.

Gate locally
Run quality checks before committing.

ROADMAP

Good next improvements

The useful path is smoother real deck work, not a bigger generic editor.

Likely next slices

Multi-deck polish
Harden selection, copying, deletion.

Media depth
Improve image-heavy slide checks.

Planning modes
Clarify high-level transformations.

Onboarding
Keep examples and guidance current.

Scope to avoid

- **No full WYSIWYG chase**
  Structured decks are the point.
- **No second renderer**
  The DOM path remains authoritative.
- **No blind apply**
  Generated changes stay reviewable.

TAKEAWAY

The basic habit

Keep the story, source, preview, review, validation, and archive in one local loop.

- **Start with intent**

  Choose a presentation, write the brief, then edit the slides that serve it.

- **Review every option**

  Use variants and deck plans as proposals, not automatic replacements.

- **Ship only checked output**

  Build, validate, refresh baselines when needed, and update the archive.

DAILY COMMANDS

Build

npm run build

Quality gate

npm run quality:gate